Microsoft® Windows

Software Development Kit

Update to Programmer's Reference and Programming Guide

Version 1.03

Microsoft Corporation

Information in this document is subject to change without notice and does not represent a commitment on the part of Microsoft Corporation. The software described in this document is furnished under a license agreement or nondisclosure agreement. The software may be used or copied only in accordance with the terms of the agreement. It is against the law to copy this software on magnetic tape, disk, or any other medium for any purpose other than the purchaser's personal use.

© Copyright Microsoft Corporation, 1986

Microsoft®, the Microsoft logo, and MS-DOS® are registered trademarks of Microsoft Corporation.

Document Number 050051011-103-I00-1086
Part Number 050-150-058

Introduction

This *Update* describes features new to Microsoft Windows operating environment version 1.03, as well as clarifies features previously documented. Each entry in this document lists a section and page number in the *Microsoft Windows Programmer's Reference* or *Microsoft Windows Programming Guide* that contains any previous information about the given function, message, statement, or data type. See the "Windows Update Directory" on pages 5–7 for an alphabetical listing of all functions, messages, statements, or data types that are further described in this *Update*.

Programmer's Reference

Windows Update Directory 5
Windows Functions 9
GDI Functions 19
System Resource Functions 37
Data Types and Structures 44
File Formats 52
Assembly-Language Macros 60
Window Messages 61

Windows Update Directory

This directory lists the Microsoft Windows 1.03 functions, messages, statements, and data types that either are new to Windows or have been more thoroughly documented in this version of the Microsoft Windows Software Development Kit. The page numbers in the directory refer to pages in this *Update* where you will find new information.

AdjustWindowRect Function	9
Bitmap Data Structure	44
CODE Statement	53
CallMsgFilter Function	10
ChangeMenu Function	16
CloseWindow Function	12
CopyRect Function	18
CreateDiscardableBitmap Function	20
CreateFont Function	20
CreateHatchBrush Function	20
DATA Statement	54
DEVICEDATA Escape Function	23
DIALOG Statement	52
DialogBox Function	13
DlgDirList Function	14
DlgDirSelect Function	14
DrawIcon Function	19
EM_GETHANDLE Message	65
EM_GETMODIFY Message	66
EM_LINELENGTH Message	66
EM_SCROLL Message	66
EM_SETHANDLE Message	65
EM_SETMODIFY Message	66
ENABLEPAIRKERNING Escape Function	24
ENABLERELATIVEWIDTHS Escape Function	25
EXTTEXTMETRIC Data Structure	45
EXTTEXTOUT Escape Function	26
EnumClipboardFormats Function	14
EnumFonts Function	35
EnumObjects Function	35
EnumWindows Function	12
EqualRgn Function	22
Extensions Section	59
ExternX Statement	60

FillRect Function	19
FrameRect Function	20
FreeProcInstance Function	37
GETEXTENDEDTEXTMETRICS Escape Function	27
GETEXTENTTABLE Escape Function	28
GETPAIRKERNTABLE Escape Function	29
GETTRACKKERNTABLE Escape Function	30
GetActiveWindow Function	12
GetAtomHandle Function	41
GetCaretBlinkTime Function	18
GetClassWord Function	11
GetCommError Function	42
GetDeviceCaps Function	36
GetProcAddress Function	37
GetProfileInt Function	42
GetScrollRange Function	17
GlobalHandle Function	38
GlobalReAlloc Function	37
GrayString Function	19
InSendMessage Function	11
InflateRect Function	18
InvertRect Function	20
IsChild Function	13
IsClipboardFormatAvailable Function	14
KERNPAIR Data Structure	49
KERNTRACK Data Structure	50
LB_RESETCONTENT Message	67
LB_SETCURSEL Message	67
LB_SETSEL Message	67
LIBRARY Statement	52
LabelX Statement	60
LineDDA Function	19
LocalHandle Function	38
LocalInit Function	39
LocalNotify Function	39
LockData Function	38
LockResource Function	41
LockSegment Function	40
MakeProcInstance Function	37
MoveWindow Function	12
NAME Statement	52
New Escape Functions	22
OffsetRect Function	18
OpenFile Function	43
PeekMessage Function	10

Polyline Function	19
Ports Section	58
SEGMENTS Statement	56
SELECTPAPERSOURCE Escape Function	31
SETCOPYCOUNT Escape Function	32
SETKERNTRACK Escape Function	33
STRETCHBLT Escape Function	34
STUB Statement	58
ScrollWindow Function	16
SelectObject Function	21
SetCaretBlinkTime Function	17
SetClassWord Function	12
SetCursor Function	17
SetRect Function	18
SetRectEmpty Function	18
SetRelAbs Function	22
SetScrollRange Function	17
SetTimer Function	15
SwapMouseButton Function	15
System Resource Functions	37
TranslateAccelerator Function	11
TranslateAccelerator Function	11
UnlockData Function	38
UnlockSegment Function	41
WM_CANCELMODE Message	62
WM_CHAR Message	63
WM_COMMAND Message	64
WM_CTLCOLOR Message	61
WM_CTLCOLOR Message	61
WM_DEADCHAR Message	63
WM_GETDLGCODE Message	62
WM_INITDIALOG Message	63
WM_KEYDOWN Message	63
WM_KEYUP Message	63
WM_SETREDRAW Message	62
WM_SIZEWAIT Message	61
WM_SYSCHAR Message	65
WM_SYSCOMMAND Message	65
WM_SYSKEYDOWN Message	64
WM_SYSKEYUP Message	65
WM_SYSTIMER Message	61
WaitSoundState Function	43
Windows Section	58
WriteComm Function	42

Windows Functions
(Chapter 2, Pages 7–104)

AdjustWindowRect Function
(Section 2.16, New)

void AdjustWindowRect(*lpRect*, *lStyle*, *bMenu***)**

This function converts the client rectangle of a window to the corresponding window rectangle. A client rectangle is the smallest rectangle that completely encloses a client area. A window rectangle is the smallest rectangle that completely encloses the window. The dimensions of the resulting window rectangle depend on the window styles and on whether the window has a menu.

Parameter	Type/Description
lpRect	**LPRECT** A long pointer to a **RECT** data structure containing the coordinates of the client rectangle. This structure also receives the coordinates of the window rectangle.
lStyle	**long** The window styles of the window whose client rectangle is to be converted.
bMenu	**BOOL** Specifies whether the window has a menu.

Return Value

None.

Comments

AdjustWindowRect typically is used to compute the required size of the window rectangle based on the desired client rectangle size. The window rectangle then can be passed to the **CreateWindow** function to create a window whose client area is the desired size.

CallMsgFilter Function
(Section 2.23, New)

BOOL CallMsgFilter(*lpMsg, nCode***)**

This function passes the given message and code to the current message-filter function. The message-filter function is an application-specified function that examines and modifies all messages. An application specifies the function using the **SetWindowsHook** function.

Parameter	Type/Description
lpMsg	**LPMSG** A long pointer to an **MSG** data structure that contains the message to be filtered.
nCode	**int** A code used by the filter function to determine how to process the message.

Return Value

The return value is TRUE if the message should not be processed further. It is FALSE if the message should be processed.

Comments

CallMsgFilter usually is called by Windows to let applications examine and control the flow of messages during internal processing in menus and scrolls bars or when moving or sizing a window.

Values given for the *nCode* parameter must not conflict with any of the MSGF_ and HC_ values passed by Windows to the message-filter function.

PeekMessage Function
(Section 2.3, Pages 12–13)

In the *Purpose* section, the phrase "returns immediately" means that **PeekMessage**, unlike **GetMessage**, does not wait for a message to be placed in the queue. Nevertheless, **PeekMessage** still yields control as part of its normal processing. This means that **PeekMessage** cannot return until Windows returns control to it.

TranslateAccelerator Function
(Section 2.3, Pages 16–17)

The return value *bTranslated* is an integer not a Boolean value.

InSendMessage Function
(Section 2.3, New)

BOOL InSendMessage( )

This function specifies whether the current window function is processing a message that is passed to it through a call to the **SendMessage** function.

Return Value

The return value is TRUE if the window function is processing a message sent to it with **SendMessage**. Otherwise, it is FALSE.

Comments

InSendMessage typically is used by applications to determine how to proceed with errors when the window that is processing the message is not the active window. For example, if the active window uses **SendMessage** to send a request for information to another window, the other window cannot become active until it returns control from the **SendMessage** call. The only method an inactive window has to inform the user of an error is to create a message box.

GetClassWord Function
(Section 2.6, Page 23)

GCW_HINSTANCE is not a valid parameter value. The value GCW_HMODULE should be used instead.

SetClassWord Function
(Section 2.6, Page 24)

GCW_HINSTANCE is not a value parameter value. The value GCW_HMODULE should be used instead.

CloseWindow Function
(Section 2.8, Page 35)

The return value *bClosed* is an integer, not a Boolean value.

MoveWindow Function
(Section 2.8, Page 35)

Any child or pop-up window has a minimum width and height. These minimums depend on the style and content of the window. Any attempt to change the width and height below the minimum using the **MoveWindow** function will fail.

EnumWindows Function
(Section 2.8, Pages 37–38)

The return value *bDone* is zero if all windows have been enumerated. Otherwise, it is nonzero.

GetActiveWindow Function
(Section 2.8, New)

GetActiveWindow ( ):*hWnd*

This function retrieves the window handle of the active window. The active window is either the window having the current input focus or the window explicitly made active by the **SetActiveWindow** function.

Parameter

None.

Return Value

hWnd is a handle to the active window.

IsChild Function
(Section 2.8, New)

BOOL IsChild(*hWndParent,hWnd***)**

This function indicates whether the window specified by *hWnd* is a child window or other direct descendant of the window specified by *hWnd-Parent*. A child window is the direct descendant of a given parent window if that parent window is in the chain of parent windows that leads from the original tiled window to the child window.

Parameter	Type/Description	
hWndParent	**HWND**	A handle to a window.
hWnd	**HWND**	A handle to the window to be checked.

Return Value

The return value is TRUE if *hWnd* is a child window of *hWndParent*. Otherwise, it is FALSE.

DialogBox Function
(Section 2.9, Pages 40–41)

The return value of the **DialogBox** function is equal to the value of the *wResult* parameter specified in the **EndDialog** function used to terminate the dialog box. Values returned by the application's dialog box are processed by Windows and are not returned to the application.

Comments

Button swapping is provided as a convenience to people who use the mouse with their left hands. **SwapMouseButton** is usually called by the Control Panel only. Although applications are free to call the function, the mouse is a shared resource and reversing the meaning of the mouse button affects all applications.

ChangeMenu Function
(Section 2.12, Pages 62–65)

The *lpNewItem* parameter is either a handle to a bitmap or a long pointer to a string. It is never a handle to a menu. The MF_POPUP flag applies to the *wIDNewItem* parameter only.

The *wIDNewItem* parameter is either a new menu item ID or a handle to a pop-up menu. It is never a menu item position. The MF_BYPOSITION and MF_BYCOMMAND flags apply to the *wIDChangeItem* parameter only.

In Table 2.3, the description of MF_MENUBARBREAK should begin as follows:

> Same as MF_MENUBREAK ....

In Table 2.3, the flags MF_INSERT, MF_BYCOMMAND, MF_ENABLED, MF_UNCHECKED, and MF_STRING are the default flags. The flags MF_CHANGE, MF_INSERT, MF_APPEND, and MF_DELETE should not be used together. The flags MF_BYPOSITION and MF_BYCOMMAND should not be used together. The flags MF_GRAYED, MF_DISABLED, and MF_ENABLED should not be used together. The flags MF_BITMAP, MF_STRING, and MF_POPUP should not be used together. MF_MENUBREAK and MF_MENUBARBREAK flags should not be used together. The MF_UNCHECKED flag and MF_CHECKED flag should not be used together.

ScrollWindow Function
(Section 2.14, Pages 77–78)

If *XAmount* is positive, the window's contents scroll to the right *not* to the left.

Comments

This function typically is called during processing of the WM_INITMENU or WM_INITMENUPOPUP message to determine whether the clipboard contains data that the application can paste. If such data are present, the application typically enables the Paste command (in its Edit menu).

This function does not cause the data to be rendered.

SetTimer Function
(Section 2.11, Page 59)

The meaning of the *nIDEvent* and *nIDNewEvent* parameters depends on the value of the *hWnd* parameter. If *hWnd* is NULL, **SetTimer** ignores *nIDEvent*, and *nIDNewEvent* is the only timer ID that can be used in the **KillTimer** function to kill the timer. If *hWnd* is not NULL, *nIDEvent* is the only timer ID that can be used to kill the timer, and *nIDNewEvent* is a Boolean value specifying whether a timer was created.

SwapMouseButton Function
(Section 2.11, New)

void SwapMouseButton(*bSwap***)**

This function reverses the meaning of left and right mouse buttons. If *bSwap* is TRUE, the left button generates right-button mouse messages and the right button generates left-button messages. If *bSwap* is FALSE, the buttons are restored to their original meaning.

Parameter	Type/Description
bSwap	**BOOL** Specifies whether the button meanings are reversed or restored.

Return Value

None.

Comments

Button swapping is provided as a convenience to people who use the mouse with their left hands. **SwapMouseButton** is usually called by the Control Panel only. Although applications are free to call the function, the mouse is a shared resource and reversing the meaning of the mouse button affects all applications.

ChangeMenu Function
(Section 2.12, Pages 62–65)

The *lpNewItem* parameter is either a handle to a bitmap or a long pointer to a string. It is never a handle to a menu. The MF_POPUP flag applies to the *wIDNewItem* parameter only.

The *wIDNewItem* parameter is either a new menu item ID or a handle to a pop-up menu. It is never a menu item position. The MF_BYPOSITION and MF_BYCOMMAND flags apply to the *wIDChangeItem* parameter only.

In Table 2.3, the description of MF_MENUBARBREAK should begin as follows:

> Same as MF_MENUBREAK ....

In Table 2.3, the flags MF_INSERT, MF_BYCOMMAND, MF_ENABLED, MF_UNCHECKED, and MF_STRING are the default flags. The flags MF_CHANGE, MF_INSERT, MF_APPEND, and MF_DELETE should not be used together. The flags MF_BYPOSITION and MF_BYCOMMAND should not be used together. The flags MF_GRAYED, MF_DISABLED, and MF_ENABLED should not be used together. The flags MF_BITMAP, MF_STRING, and MF_POPUP should not be used together. MF_MENUBREAK and MF_MENUBARBREAK flags should not be used together. The MF_UNCHECKED flag and MF_CHECKED flag should not be used together.

ScrollWindow Function
(Section 2.14, Pages 77–78)

If *XAmount* is positive, the window's contents scroll to the right *not* to the left.

SetScrollRange Function
(Section 2.14, Pages 80–81)

A scroll-bar control cannot be hidden by making the minimum and maximum scroll range positions equal. Although this technique can be applied to a scroll-bar control, it disables the control but does not hide it. Only standard scroll bars, accessed using SB_HORZ or SB_VERT, can be hidden in this way.

GetScrollRange Function
(Section 2.14, Page 81)

Under *Parameter*, the description of *nBar* should read as follows:

> *nBar* is one of the following short integer values, specifying which scroll bar to get:

Value	Meaning
SB_VERT	Get the vertical scroll-bar position.
SB_HORZ	Get the horizontal scroll-bar position.
SB_CTL	Get a scroll-bar control; in this case, the *hWnd* parameter specifies the control.

SetCursor Function
(Section 2.18, Page 91)

Any application that needs to change the shape of the system cursor while it is in a window must make sure the class cursor for the given window's class is set to NULL. If the class cursor is not NULL, Windows restores the old shape each time the mouse moves.

SetCaretBlinkTime Function
(Section 2.19, Page 95)

The *nMSeconds* parameter is an unsigned integer (**WORD**), not an integer.

GetCaretBlinkTime Function
(Section 2.19, Page 95)

The return value *nMSeconds* is an unsigned integer (**WORD**), not an integer.

SetRect Function
(Section 2.21, Page 97)

Although the **SetRect** function return type is an integer, the return value is not used and has no meaning.

SetRectEmpty Function
(Section 2.21, Page 97)

Although the **SetRectEmpty** function return type is an integer, the return value is not used and has no meaning.

CopyRect Function
(Section 2.21, Page 97)

Although the **CopyRect** function return type is an integer, the return value is not used and has no meaning.

InflateRect Function
(Section 2.21, Page 98)

Although the **InflateRect** function return type is an integer, the return value is not used and has no meaning.

OffsetRect Function
(Section 2.21, Page 99)

Although the **OffsetRect** function return type is an integer, the return value is not used and has no meaning.

GDI Functions
(Chapter 3, Pages 105–188)

Polyline Function
(Section 3.3, Pages 111–112)

This function uses the currently selected pen to draw line segments.

GrayString Function
(Section 3.3, Pages 121–122)

This function requires that you use the MM_TEXT mapping mode.

DrawIcon Function
(Section 3.3, Page 122)

This function requires that you use the MM_TEXT mapping mode.

LineDDA Function
(Section 3.3, Pages 124–125)

Use the **MakeProcInstance** function to create the address passed as the *lpLineFunc* parameter to **LineDDA**.

Your application-supplied function must use the Pascal calling convention and must be declared **FAR**.

FillRect Function
(Section 3.3, Page 126)

When filling the specified rectangle, **FillRect** does not include the rectangle's right and bottom sides. GDI fills a rectangle up to, but does not include, the right column and bottom row.

The **FillRect** function compares the values of the *top*, *bottom*, *left*, and *right* fields of the specified rectangle. If *bottom* is less than or equal to *top*, or if *right* is less than or equal to *left*, the rectangle is not drawn.

FrameRect Function
(Section 3.3, Pages 126–127)

The **FrameRect** function compares the values of the *top*, *bottom*, *left*, and *right* fields of the specified rectangle. If *bottom* is less than or equal to *top*, or if *right* is less than or equal to *left*, the rectangle is not drawn.

InvertRect Function
(Section 3.3, Page 127)

The **InvertRect** function compares the values of the *top*, *bottom*, *left*, and *right* fields of the specified rectangle. If *bottom* is less than or equal to *top*, or if *right* is less than or equal to *left*, the rectangle is not drawn.

CreateHatchBrush Function
(Section 3.4, Page 130)

The HS_FDIAGONAL value specifies a 45-degree downward hatch from left to right. The HS_BDIAGONAL value specifies a 45-degree upward hatch from left to right.

CreateFont Function
(Section 3.4, Pages 134–137)

The string pointed to by *lpFacename* must not exceed 30 characters.

(The **CreateFont** function does not create a new font, it merely selects the closest match from the fonts available in GDI's pool of physical fonts. See Appendix B, "Creating a Font Resource," in the *Microsoft Windows Programming Guide* for information about creating and adding a font to your Windows application.)

CreateDiscardableBitmap Function
(Section 3.4, New)

HBITMAP CreateDiscardableBitmap (*hDC*, *nWidth*, *nHeight*)

This function creates a bitmap in the same way as the **CreateCompatibleBitmap** function does. However, bitmaps created using the **CreateDiscardableBitmap** function are discardable and may be removed along with other discardable resources if Windows requires additional global memory.

Parameter	Type/Description
hDC	**HDC** A handle to a display context.
nWidth	**short** A short integer value that specifies the width in bits of the bitmap.
nHeight	**short** A short integer value that specifies the height in bits of the bitmap.

Return Value

The return value is a handle to a bitmap if the function is successful. Otherwise, it is NULL.

SelectObject Function
(Section 3.5, Pages 138–139)

When you select a font, pen, or brush with the **SelectObject** function, GDI allocates space for that object in its data segment. Because data segment space is limited, you should delete each drawing object that you no longer need using the **DeleteObject** function.

Upon deleting the last of the unneeded drawing objects, you should select the original (default) object back into the display context. The following example shows one way of doing this:

Example

```
                    .
                    .
                    .
hPen=CreatePen(0,1,RGB(255,0,0));          /* create red pen */

hOldPen=SelectObject(hDC,hPen);            /* select red pen */
                                           /* save default pen */

MoveTo(hDC,10,15);                         /* 10,15 becomes  */
                                           /* current position */

LineTo(hDC,100,15);                        /* draw horizontal line */
                                           /* using red pen */

DeleteObject(SelectObject(hDC,hOldPen)); /* select default pen */
                                           /* delete red pen */
                    .
                    .
                    .
```

SetRelAbs Function
(Section 3.6, Page 142)

Although the entry point for this function exists, it has not been implemented.

EqualRgn Function
(Section 3.8, Page 163)

Although the entry point for this function exists, it has not been implemented.

New Escape Functions
(Section 3.11, Page 180)

You can use the following functions to enhance device driver performance, access device-specific features, control text quality, and control text appearance. See the *Microsoft Windows Programmer's Reference*, Section 3.11, pages 171–172, for more information about the format of **Escape** functions.

The **Escape** functions are listed alphabetically by escape name, as follows:

DEVICEDATA

ENABLEPAIRKERNING

ENABLERELATIVEWIDTHS

EXTTEXTOUT

GETEXTENDEDTEXTMETRICS

GETEXTENTTABLE

GETPAIRKERNTABLE

GETTRACKKERNTABLE

SELECTPAPERSOURCE

SETCOPYCOUNT

SETKERNTRACK

STRETCHBLT

Escape Function
(Section 3.11, New)

short Escape (*hDC*, DEVICEDATA, *nCount*, *lpInData*, *lpOutData*)

This function allows the application to send data directly to the printer, bypassing the standard print-driver code.

Parameter	Type/Description
hDC	**HDC** A handle to the printer display context.
nCount	**short** The number of bytes of device data to be passed to the printer.
lpInData	**LPSTR** A structure whose first word (16-bits) contains the number of bytes of input data. The remaining bytes of the structure contain the data itself.
lpOutData	**LPSTR** Not used and can be set to NULL.

Return Value

The return value is the number of bytes transferred to the printer if the function is successful; 0 if not or if the escape is not implemented. If the returned value is nonzero but less than *nCount*, an error prohibited transmission of the entire data block.

Comments

There may be restrictions on the kinds of device data an application may send to the device without interfering with the operation of the driver. In general, applications must avoid resetting the printer or causing the page to be printed. Additionally, applications are strongly discouraged from performing functions that consume printer memory, for example, downloading a font or a macro.

The driver should invalidate its internal state variables such as "current position," "current line style," etc., when executing this escape. The driver may issue a printer "save" command prior to transmitting the first of a sequence of DEVICEDATA escapes and issue a "restore" command prior to executing the first command after the last DEVICEDATA escape. In other words, the application must be able to issue multiple, sequential DEVICEDATA escapes without intervening "saves" and "restores" inserted by the driver.

Escape Function
(Section 3.11, New)

BOOL Escape (*hDC*, ENABLEPAIRKERNING, *nCount*, *lpInData*, *lpOutData*)

This function enables or disables the driver's ability to kern character pairs automatically. When enabled, the driver automatically will kern those pairs of characters that are listed in the font's character-pair kerning table. The driver will reflect this kerning both on the printer and in **GetTextExtent** calls.

Parameter	Type/Description
hDC	**HDC** A handle to the printer display context.
nCount	**short** The number of bytes used by *lpInData*. This value should be set to 2.
lpInData	**LPSTR** A long pointer to a short integer value that specifies whether automatic pair kerning is to be enabled (1) or disabled (zero).
lpOutData	**LPSTR** A long pointer to a short integer variable that will receive the previous automatic-pair-kerning flag.

Return Value

The return value is 1 if the function is successful; zero if not or if the escape is not implemented.

Comments

The default state of this capability is zero, that is, automatic character-pair kerning is disabled.

A driver does not have to support this escape just because it supplies the character-pair kerning table to the application via the GETPAIR-KERNTABLE escape. In the case where the GETPAIRKERNTABLE escape is supported but the ENABLEPAIRKERNING escape is not, it is the application's responsibility to properly space the kerned characters on the output device.

Escape Function
(Section 3.11, New)

BOOL Escape (*hDC*, ENABLERELATIVEWIDTHS, *nCount*, *lpInData*, *lpOutData*)

This function enables or disables relative character widths. When disabled (the default), each character's width can be expressed as an integer number of device units. This guarantees that the extent of a string will equal the sum of the extents of the characters in the string. Such behavior allows applications to manually build an extent table using one-character **GetTextExtent** calls. When enabled, the sum of a string may not equal the sum of the widths of the characters. Applications that enable this feature are expected to retrieve the font's extent table and compute relatively-scaled string widths themselves.

Parameter	Type/Description
hDC	**HDC** A handle to the printer display context.
nCount	**short** The number of bytes used by *lpInData*. This value should be set to 2.
lpInData	**LPSTR** A long pointer to a short integer value that specifies whether relative widths are to be enabled (1) or disabled (zero).
lpOutData	**LPSTR** A long pointer to a short integer variable that will receive the previous relative-character-width flag.

Return Value

The return value is 1 if the function is successful; zero if not or if the escape is not implemented.

Comments

The default state of this capability is zero, that is, relative character widths are disabled.

Enabling this feature causes values specified as "font units" and accepted and returned by the escapes described in this section to be returned in the relative units of the font.

It is assumed that only linear scaling devices will be dealt with in a relative mode. Non-linear scaling devices should not implement this escape.

It has not yet been decided whether this capability will ever become part of Windows GDI.

Escape Function
(Section 3.11, New)

BOOL Escape (*hDC*, EXTTEXTOUT, *nCount*, *lpInData*, *lpOutData*)

This function provides a more efficient way for the application to invoke the GDI **TextOut** function when justification, letterspacing, and/or kerning is involved.

Parameter	Type/Description
hDC	**HDC** A handle to the printer display context.
nCount	**short** The number of bytes used by *lpInData*.
lpInData	**LPSTR** A long pointer to a data structure described in the Comments section below.
lpOutData	**LPSTR** Not used and can be set to NULL.

Return Value

The return value is 1 if the function is successful; zero if not or if the escape is not implemented.

Comments

The structure pointed to by *lpIndata* contains the following items:

```
WORD X, Y;
LPSTR cch;
RECT clipRect;
WORD lpText;
WORD FAR *lpWidths;
```

The *X* argument specifies the *x*-coordinate of the upper-left corner of the string's starting point. The *Y* argument specifies the *y*-coordinate of the upper-left corner of the string's starting point. The *lpText* argument is a long pointer to an array of *cch* character codes. The *cch* argument is the

number of bytes in the string (*cch* is also the number of words in the width array). The *lpWidths* argument is a long pointer to an array of *cch* character widths to use when printing the string. The first character appears at (*X*, *Y*), the second at (*X*+*lpWidths*[0], *Y*), the third at (*X*+ *lpWidths*[0]+ *lpWidths*[1], *Y*), and so on. These character widths are specified in the font units of the currently selected font. (The character widths will always be equal to device units unless the application has enabled relative character widths.)

The units contained in the width array are specified as font units of the device (see the previous EXTTEXTMETRIC escape) and are dependent on whether relative character widths are enabled (see the previous ENABLE-RELATIVEWIDTHS escape).

Escape Function
(Section 3.11, New)

short Escape (*hDC*, GETEXTENDEDTEXTMETRICS, *nCount*, *lpInData*, *lpOutData*)

This function fills the buffer pointed to by *lpOutData* with the extended text metrics for the currently selected font.

Parameter	Type/Description
hDC	**HDC** A handle to the printer display context.
nCount	**short** Not used and can be set to NULL.
lpInData	**LPSTR** A long pointer to a data structure described in the following Comments section.
lpOutData	**LPSTR** A long pointer to a data structure of type **EXTTEXTMETRIC**.

Return Value

The return value is the number of bytes copied to the buffer pointed to by *lpOutData*. This value will never exceed *nSize* and will be zero if the function fails or the escape is not implemented.

Comments

The structure pointed to by *lpIndata* contains the following item:

```
WORD nSize;
```

The *nSize* parameter contains the number of bytes pointed to by *lpOutData*.

The values returned in many of the fields of the **EXTTEXTMETRIC** structure are affected by whether relative character widths are enabled or disabled. See the preceding ENABLERELATIVEWIDTHS escape.

Escape Function
(Section 3.11, New)

BOOL Escape (*hDC*, GETEXTENTTABLE, *nCount*, *lpInData*, *lpOutData*)

This function returns the width (extent) of individual characters from a group of consecutive characters in the selected font's character set. The first and last character (from the group of consecutive characters) are function arguments.

Parameter	Type/Description
hDC	**HDC** A handle to the printer display context.
nCount	**short** Not used and can be set to NULL.
lpInData	**LPSTR** A long pointer to a data structure described below in the following Comments section.
lpOutData	**LPSTR** A long pointer to an array of type **short**. The size of the array must be at least (*chLast* − *chFirst* + 1).

Return Value

The return value is 1 if the function is successful; zero if it is not or if the escape is not implemented.

Comments

The structure pointed to by *lpInData* contains the following items:

```
BYTE chFirst;
BYTE chLast;
```

The *chFirst* argument contains the character code of the first character. The *chLast* argument contains the character code of the last character.

The values returned are affected by whether relative character widths are enabled or disabled. See the preceding ENABLERELATIVEWIDTHS escape.

Escape Function
(Section 3.11, New)

short Escape (*hDC*, GETPAIRKERNTABLE, *nCount*, *lpInData*, *lpOutData*)

This function fills the buffer pointed to by *lpOutData* with the character-pair kerning table for the currently selected font.

Parameter	Type/Description
hDC	**HDC** A handle to the printer display context.
nCount	**short** Not used and can be set to NULL.
lpInData	**LPSTR** Not used and can be set to NULL.
lpOutData	**LPSTR** A long pointer to an array of **KERNPAIR** structures. This array must be large enough to accomodate the font's entire character-pair kerning table. The number of character-kerning pairs in the font can be obtained from the **EXTTEXTMETRIC** structure returned by the GETEXTENDEDTEXT-METRICS escape.

Return Value

The return value is the number of **KERNPAIR** structures copied to the buffer. This value is zero if the font does not have kerning pairs defined, the function fails, or the escape is not implemented.

Comments

The values returned in the **KERNPAIR** structures are affected by whether relative character widths are enabled or disabled. See the preceding ENABLERELATIVEWIDTHS escape.

Escape Function
(Section 3.11, New)

short Escape (*hDC*, GETTRACKKERNTABLE, *nCount*, *lpInData*, *lpOutData*)

This function fills the buffer pointed to by *lpOutData* with the track-kerning table for the currently selected font.

Parameter	Type/Description
hDC	**HDC** A handle to the printer display context.
nCount	**short** Not used and can be set to NULL.
lpInData	**LPSTR** Not used and can be set to NULL.
lpOutdata	**LPSTR** A long pointer to an array of **KERNTRACK** structures. This array must be large enough to accomodate all the font's kerning tracks. The number of tracks in the font can be obtained from the **EXTTEXTMETRIC** structure returned by the GETEXTENDEDTEXTMETRICS escape.

Return Value

The return value is the number of **KERNTRACK** structures copied to the buffer. This value is zero is the font does not have kerning tracks defined, the function fails, or the escape is not implemented.

Comments

The values returned in the **KERNTRACK** structures are affected by whether relative character widths are enabled or disabled. See the preceding ENABLERELATIVEWIDTHS escape.

Escape Function
(Section 3.11, New)

short Escape (*hDC*, SELECTPAPERSOURCE, *nCount*, *lpInData*, *lpOutData*)

This function allows the application to determine the available paper sources and select among them.

Parameter	Type/Description
hDC	**HDC** A handle to the printer display context.
nCount	**short** The number of bytes used by *lpInData*. This value should be set to 2.
lpInData	**LPSTR** A long pointer to a short integer value containing the desired paper-source index. Index 0 specifies manual feed, if available. Index 1 specifies the primary or only paper tray. Indices greater than 1 refer to additional paper sources.
lpOutData	**LPSTR** A long pointer to a data structure described in the Comments section below.

Return Value

The return value is 1 if the function is successful, –1 if the requested paper source does not exist, or zero if the escape is not implemented. The data structure pointed to by *lpOutData* is valid only if the function returns 1.

Comments

The structure pointed to by *lpOutData* contains the following items:

```
WORD X, Y;
RECT imageRect;
WORD orientation;
```

The *X* argument specifies the horizontal paper size in device units. The *Y* argument specifies the vertical paper size in device units. The *imageRect* argument specifies the size of the rectangular region on a page that can be printed on. The *orientation* argument specifies whether the page orientation is portrait (1) or landscape (2).

The application may enumerate the available paper sources by invoking this escape with indices 0, 1, 2, etc., after which the escape returns a 1.

For the manual feed index (0), the returned values refer to the maximum paper size that can be fed manually. It is the user's responsibility to manually feed the proper size paper.

The driver defaults to the paper-source index (1), the primary paper tray. The driver will reset to this value if it returns a –1 because of an illegal paper-source request.

Paper sources can be changed only between frames.

Escape Function
(Section 3.11, New)

short Escape (*hDC*, SETCOPYCOUNT, *nCount*, *lpInData*, *lpOutData*)

This function specifies the number of uncollated copies of each page that the printer is to print.

Parameter	Type/Description
hDC	**HDC** A handle to the printer display context.
nCount	**short** The number of bytes used by *lpInData*. This value should be set to 2.
lpInData	**LPSTR** A long pointer to a short integer value containing the number of uncollated copies to be printed.
lpOutData	**LPSTR** A long pointer to a short integer variable that will receive the number of copies to be printed. This may be less than the number requested if the requested number is greater than the device's maximum copy count.

Return Value

The return value is 1 if the function is successful; zero if not or if the escape is not implemented.

Escape Function
(Section 3.11, New)

BOOL Escape (*hDC*, SETKERNTRACK, *nCount*, *lpInData*, *lpOutData*)

For drivers that support automatic track kerning, this escape specifies which kerning track will be used. A kerning track of zero disables automatic track kerning. When enabled, the driver will automatically kern all characters according to the specified track. The driver will reflect this kerning both on the printer and in **GetTextExtent** calls.

Parameter	Type/Description
hDC	**HDC** A handle to the printer display context.
nCount	**short** The number of bytes used by *lpInData*. This value should be set to 2.
lpInData	**LPSTR** A long pointer to a short integer value that specifies the kerning track to use. A value of zero disables this feature. Values 1 to *nKernTracks* (see the **EXTTEXTMETRIC** structure in the "Data Types and Structures" section of this Update) correspond to positions in the track-kerning table (using 1 as the first item in the table).
lpOutData	**LPSTR** A long pointer to a short integer variable that will receive the previous kerning track.

Return Value

The return value is 1 if the function is successful; zero if not or if the escape is not implemented.

Comments

The default state of this capability is zero, that is, automatic track kerning is disabled.

A driver does not have to support this escape just because it supplies the track-kerning table to the application using the GETTRACKKERNTABLE escape. In the case where GETTRACKKERNTABLE is supported but the SETKERNTRACK escape is not, it is the application's responsibility to properly space the characters on the output device.

Escape Function
(Section 3.11, New)

BOOL Escape (*hDC*, STRETCHBLT, *nCount*, *lpInData*, *lpOutData*)

This function implements the GDI function **StretchBlt** on the driver level.

Parameter	Type/Description
hDC	**HDC** A handle to the printer display context.
nCount	**short** The number of bytes used by *lpInData*.
lpInData	**LPSTR** A long pointer to a data structure described below in the Comments section.
lpOutData	**LPSTR** Not used and can be set to NULL.

Return Value

The return value is 1 if the function is successful; zero if not or if the escape is not implemented.

Comments

The data structure pointed to by *lpInData* contains the following items:

```
WORD X, Y;
WORD nWidth, nHeight;
WORD XSrc, YSrc;
WORD nSrcWidth;
WORD nSrcHeight;
DWORD dwRop;
```

```
short bmType;
short bmWidth;
short bmHeight;
short bmWidthBytes;
short bmPlanes;
short bmBitsPixel;
short bmBits;
```

The members of this structure correspond to the arguments to the **StretchBlt** function described on page 117 of the *Microsoft Windows Programmer's Reference*. The *bmBits* member of this **BITMAP** structure is a long pointer to the bitmap bits. This pointer must be set by the application. This escape should be implemented only for devices with bitmap-stretching capabilities.

The driver need not implement all of GDI's raster operations, but must return zero if the specific ROP (raster operation) cannot be implemented. The driver must not substitute one ROP for another.

EnumFonts Function
(Section 3.12, Pages 180–181)

Use the **MakeProcInstance** function to create the address passed as the *lpFontFunc* parameter to **EnumFonts**.

Your application-supplied function must use the Pascal calling convention and must be declared **FAR**.

EnumObjects Function
(Section 3.12, Pages 181–182)

Use the **MakeProcInstance** function to create the address passed as the *lpLineFunc* parameter to **EnumObjects**.

Your application-supplied function must use the Pascal calling convention and must be declared **FAR**.

GetDeviceCaps Function
(Section 3.12, Pages 183–185)

Table 3.3 should contains the following additional information indexes:

Index	Meaning
LOGPIXELSX	The number of pixels per logical inch along the display width.
LOGPIXELSY	The number in pixels per logical inch along the display height.

For details about the logical screen width and height, see Appendix C, "GDI Definitions," in the *Microsoft Windows Programming Guide.*

System Resource Functions
(Chapter 4, Pages 189–248)

GetProcAddress Function
(Section 4.2, Pages 192–193)

The *lpFilename* parameter can specify either a long pointer to the function name or the ordinal value of the function. If it is an ordinal value, the value must be in the low-order word and zero must be in the high-order word.

MakeProcInstance Function
(Section 4.2, Pages 193–194)

Once a procedure instance has been created, subsequent calls to the **MakeProcInstance** function using the same function and instance handle return the same procedure instance address. In other words, no more than one procedure instance can be created for any one function in any given instance of an application.

FreeProcInstance Function
(Section 4.2, Page 194)

After a procedure instance is freed, subsequent use of the procedure instance address will result in serious error.

GlobalReAlloc Function
(Section 4.3, Pages 198–199)

Under the GMEM_MODIFY description, the third sentence should read as follows:

The *dwBytes* parameter is ignored.

LockData/UnlockData Functions
(Section 4.3, Page 205)

The **LockData** and **UnlockData** functions are implemented as macros and require that the *windows.h* file be included in the program source.

GlobalHandle Function
(Section 4.3, New)

GlobalHandle(*wMem***):***dwMem*

This function retrieves the handle of the global memory object whose segment address is *wMem*.

Parameter	Type/Description
wMem	**WORD** An unsigned integer value that specifies the segment address of a global memory object.

Return Value

The *dwMem* parameter is an unsigned long integer (**DWORD**) whose low-order word contains the handle of the global memory object and whose high-order word contains the segment address of the memory object.

LocalHandle Function
(Section 4.3, New)

LocalHandle(*wMem***):***hMem*

This function retrieves the handle of the local memory object whose address is *wMem*.

Parameter	Type/Description
wMem	**WORD** An unsigned integer value that specifies the address of a local memory object.

Return Value

The *hMem* parameter is the handle of the local memory object.

LocalInit Function
(Section 4.3, New)

LocalInit(*wSegment, wStart, wEnd*): *bResult*

This function initializes a local heap in the segment specified by *wSegment*.

Parameter	Type/Description
wSegment	**WORD** An unsigned integer value that specifies the segment address of the segment to contain the local heap.
wStart	**WORD** An unsigned integer value that specifies the address of the start of the local heap within the segment.
wEnd	**WORD** An unsigned integer value that specifies the address of the end of the local heap within the segment.

Return Value

The *bResult* parameter, a Boolean value, is nonzero if the heap is initialized. Otherwise, it is zero.

Comments

If *wStart* is zero, the default starting position for the local heap is *wEnd* bytes from the end of the given segment.

LocalNotify Function
(Section 4.3, New)

LocalNotify(*lpNotifyFunc*):*lpPrevFunc*

This function sets the address of the local heap notification handler. The local heap notification handler processes notification messages generated by Windows when certain actions occur, such as a request to increase the size of the heap.

Parameter	Type/Description
lpNotifyFunc	**FARPROC** A long pointer to the notification message handler.

Return Value

The *lpPrevFunc* parameter is a long pointer to the previous notification handler.

Comments

The address passed as the *lpNotifyFunc* parameter must be created using the **MakeProcInstance** function.

The local heap notification handler must have the following form:

BOOL FAR PASCAL *NotifyFunc*(*wMsg*,*hMem*,*wArg*)
WORD *wMsg*;
HANDLE *hMem*;
WORD *wArg*;

The *wMsg* parameter specifies the notification message. The *hMem* parameter is the handle of the local memory object that generated the notification. The *wArg* parameter is an argument whose value depends on the value of *wMsg*. The function should return nonzero if it is successful.

LockSegment Function
(Section 4.3, New)

LockSegment(*wSegment*): *hSegment*

This function locks the segment whose segment address is *wSegment*. If *wSegment* is –1, **LockSegment** locks the current data segment.

Parameter	Type/Description
wSegment	**WORD** An unsigned integer that specifies the segment address of the segment to be locked.

Return Value

The *hSegment* parameter is the global memory handle of the locked segment.

UnlockSegment Function
(Section 4.3, New)

UnlockSegment(*wSegment*): *hSegment*

This function unlocks the segment whose segment address is *wSegment*. If *wSegment* is –1, **LockSegment** unlocks the current data segment.

Parameter	Type/Description
wSegment	**WORD** An unsigned integer that specifies the segment address of the segment to be unlocked.

Return Value

The *hSegment* parameter is the global memory handle of the unlocked segment.

LockResource Function
(Section 4.5, Pages 214–215)

The *hResInfo* parameter is the resource handle associated with a *loaded* resource. This is the handle returned by the **LoadResource** function, *not* the **FindResource** function. Using the handle returned by the **Find-Resource** function for this parameter causes an error.

GetAtomHandle Function
(Section 4.7, New)

GetAtomHandle(*wAtom*): *hMem*

This function retrieves a handle (relative to the local heap) of the string that corresponds to the atom specified by *wAtom*.

Parameter	Type/Description
wAtom	**WORD** An unsigned integer that specifies the atom whose handle is to be retrieved.

Return Value

The *hMem* parameter is a handle to the given atom's string. It is zero if no such atom exists.

GetProfileInt Function
(Section 4.8, Page 222)

The **GetProfileInt** function returns zero, instead of the default value, if the value corresponding to the specified keyname is not an integer. If the value corresponding to the keyname consists of digits followed by non-numeric characters, the function returns the value of the digits. For example, if the entry "KeyName=102abc" is accessed, the function returns 102.

WriteComm Function
(Section 4.10, Page 227)

The **WriteComm** function will overwrite data in the transmit queue if there is not enough room in the queue for the additional characters. Applications should check the available space in the transmit queue with the **GetCommError** function before calling **WriteComm**. Also, applications should use **OpenComm** to set the size of the transmit queue to an amount no smaller than the size of the largest expected output string.

GetCommError Function
(Section 4.10, Pages 229–230)

In case of a communications error, Windows locks the communications port until the error is cleared using the **GetCommError** function.

In Table 4.1, the code CE_RXOVERRUN should be CE_RXOVER.

WaitSoundState Function
(Section 4.11, Pages 240–241)

In the *Parameters* section, the values S_QUEUEEMPTY, S_THRESHOLD, and S_ALLTHRESHOLD should be QUEUEEMPTY, THRESHOLD, and ALLTHRESHOLD.

OpenFile Function
(Section 4.13, Page 245–246)

If wildcard characters are used in the file specification, the **OpenFile** function searches all directories in the PATH environment variable for a matching filename.

Style values can be combined using the bitwise OR operator, not the logical OR operator.

The OF_CREATE flag always directs **OpenFile** to create a new file. If the file already exists, it is truncated to zero length.

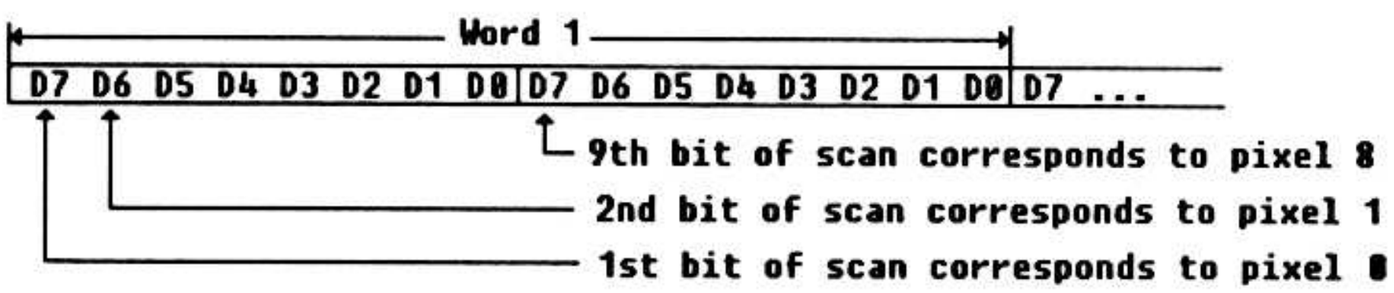
Data Types and Structures
(Chapter 5, Pages 249–278)

Bitmap Data Structure
(Section 5.3, Page 265)

The last sentence in the discussion of the *bmWidthBytes* field states that *bmWidthBytes*8* must be the next multiple of eight greater than or equal to *bmWidth*. This is incorrect; *bmWidthBytes*8* must be the next multiple of sixteen (*not* eight) greater than or equal to *bmWidth*.

The *bmBits* field can be further defined as follows:

The currently used bitmap formats are monochrome and color. The monochrome bitmap uses a one-bit, one-plane format. Each scan is a multiple of sixteen bits. The following figure shows the relationship between bits in the bitmap and pixels on the display:



Scans are organized as follows for a bitmap of height *n*:

```
Scan 0
Scan 1
   .
   .
   .
Scan n-2
Scan n-1
```

The pixels on a monochrome device are either black or white. If the corresponding bit in the bitmap is 1, the pixel is turned on (white); if the corresponding bit in the bitmap is zero, the pixel is turned off (black).

All devices that have the RC_BITBLT bit set in the device capabilities support bitmaps.

The color bitmap uses a one-bit, three-plane format. This format applies to high resolution and EGA high and low resolution drivers. Each scan is a multiple of sixteen bits. The preceding figure shows the relationship between bits in the bitmap and pixels on a color plane.

Scans are organized as follows for a bitmap of height *n*:

```
Red     Scan 0
Red     Scan 1
  .
  .
  .
Red     Scan n-2
Red     Scan n-1
Green   Scan 0
Green   Scan 1
  .
  .
  .
Green   Scan n-2
Green   Scan n-1
Blue    Scan 0
Blue    Scan 1
  .
  .
  .
Blue    Scan n-2
Blue    Scan n-1
```

Pixels in a color plane correspond to bits in the bitmap as follows:

```
0 = color disabled
1 = color enabled
```

EXTTEXTMETRIC Data Structure
(Section 5.3, New)

EXTTEXTMETRIC - *Extended Font Metrics*

The **EXTTEXTMETRIC** structure is a list of the extended metrics of a physical font. The structure is returned by the GETEXTENDEDTEXT-METRICS escape.

The **EXTTEXTMETRIC** structure contains the following fields:

```
short etmSize
short etmPointSize
short etmOrientation
short etmMasterHeight
short etmMinScale
short etmMaxScale
short etmMasterUnits
short etmCapHeight
short etmXHeight
short etmLowerCaseAscent
short etmUpperCaseDescent
short etmSlant
short etmSuperScript
short etmSubScript
short etmSuperScriptSize
short etmSubScriptSize
short etmUnderlineOffset
short etmUnderlineOffset
short etmUnderlineWidth
short etmDoubleUpperUnderlineOffset
short etmDoubleLowerUnderlineOffset
short etmDoubleUpperUnderlineWidth
short etmDoubleLowerUnderlineWidth
short etmStrikeOutOffset
short etmStrikeOutWidth
short etmNKernPairs
short etmNKernTracks
```

The **EXTTEXTMETRIC** fields are described below. All sizes are given in device units (i.e., they do not depend on the current mapping mode of the display context).

Field	Definition
etmSize	Specifies the size of the **EXTTEXTMETRIC** structure in bytes. The addition of new fields to the **EXTTEXTMETRIC** structure will not alter the position of existing fields. The *etmSize* field and the *nSize* parameter of the GETEXTENDEDTEXTMETRICS escape will reflect the addition of new fields to the **EXTTEXTMETRIC** structure.
etmPointSize	Specifies the font's point size in twips (twentieths of a point). The font will not appear in *etmPointSize* on the device if the size of device pixels isn't a multiple of a twip.

etmOrientation	Specifies the font's orientation: 1 if portrait, 2 if landscape, zero if either orientation. This value reflects whether GDI can produce the font in the specified orientation(s). Landscape pages are defined as pages whose width (*x*-dimension) is greater than their height (*y*-dimension); pages that do not meet these requirements are defined as portrait.
etmMasterHeight	Specifies the font height in device units. On linear scaling devices this may not be the same as the requested size of the font.
etmMinScale	Specifies the minimum range in device units for the font.
etmMaxScale	Specifies the maximum range in device units for the font.
etmMasterUnits	Specifies the integer number of units per em of the *MasterSize*. (An em is the width of the uppercase "M" for the particular font.) If the original width and height are already integers, the *MasterSize* is the original height. However, if either the width or height is a rational number, both numbers must be multiplied by some integer value to produce integer results. For example, if the font width were 4.5 points and the font height were 12 points, you would multiply both numbers by two to create an integer value of 9 for the width and 24 for the height. The *MasterSize* for the font would be the resultant height value (24). If this value is not equal to 1, then the device is a relative-scaling device and the device's font units are relative units. The application is responsible for scaling the relative units according to the following equation: `ScaledValue=Value*UserSize/MasterUnits`
etmCapHeight	Specifies the height in font units of uppercase characters in the font, typically, the height of the capital "H."
etmXHeight	Specifies the height in font units of lowercase characters in the font, typically, the height of the lowercase "x."

etmLowerCaseAscent	Specifies the distance in font units that the ascenders of lowercase letters extend above the baseline, typically specified for a lowercase "d."
etmLowerCaseDescent	Specifies the distance in font units that the descenders of lowercase letters extend below the baseline, typically specified for a lowercase "p."
etmSlant	Specifies the angle in degrees counterclockwise from the vertical axis.
etmSuperScript	Specifies the distance from the origin of the superscript to the baseline of the regular font. This value is specified as a negative offset.
etmSubScript	Specifies the distance from the origin of the subscript to the baseline of the regular font. This value is specified as a positive offset.
etmSuperScriptSize	Specifies the recommended size of superscripts in the selected font.
etmSubScriptSize	Specifies the recommended size of subscripts in the selected font.
etmUnderlineOffset	Specifies the distance from the baseline to the top of a single underline.
etmUnderlineWidth	Specifies the thickness of the underline.
etmDoubleUpper-UnderlineOffset	Specifies the distance from the baseline to the top of the double underline.
etmDoubleUpper-UnderlineWidth	Specifies the thickness of the upper underline in double underline.
etmDoubleLower-UnderlineWidth	Specifies the thickness of the lower underline in a double underline.
etmStrikeOutOffset	Specifies the distance from the baseline to the top of a strike-out line.
etmNKernPairs	Specifies the integer number of kerned character pairs defined for the selected font. This value represents the number of kerned pairs listed in the character-pair kerning table. The kerning table is produced by the GETPAIRKERNTABLE escape.

etmNKernTracks	Specifies the integer number of kerning tracks defined for the selected font. This value represents the number of kern tracks listed in the table. The track-kerning table is produced by the GETTRACKKERNTABLE escape.

KERNPAIR Data Structure
(Section 5.3, New)

The **KERNPAIR** structure contains two characters that are kerned if they appear side by side in a particular font. The structure also contains the amount of kerning that will occur.

The **KERNPAIR** structure contains the following fields:

```
union {
        BYTE each[2];
        WORD both;
      } pair;
        short kernAmount;
```

The **KERNPAIR** fields are described below. All measurements are given in the font units of the device regardless of the current mapping mode of the display context.

Field	Definition
pair.each[0]	The character code for the first character in the kerning pair.
pair.each[1]	The character code for the second character in the kerning pair.
pair.both	A **WORD** type that contains the first character of the kerning pair in the low-order byte and the second character in the high-order byte.
kernAmount	The signed amount that this pair will be kerned if it appears side-by-side in the same font and size. This value typically is negative since pair kerning usually results in two characters being set more tightly than normal.

KERNTRACK Data Structure
(Section 5.3, New)

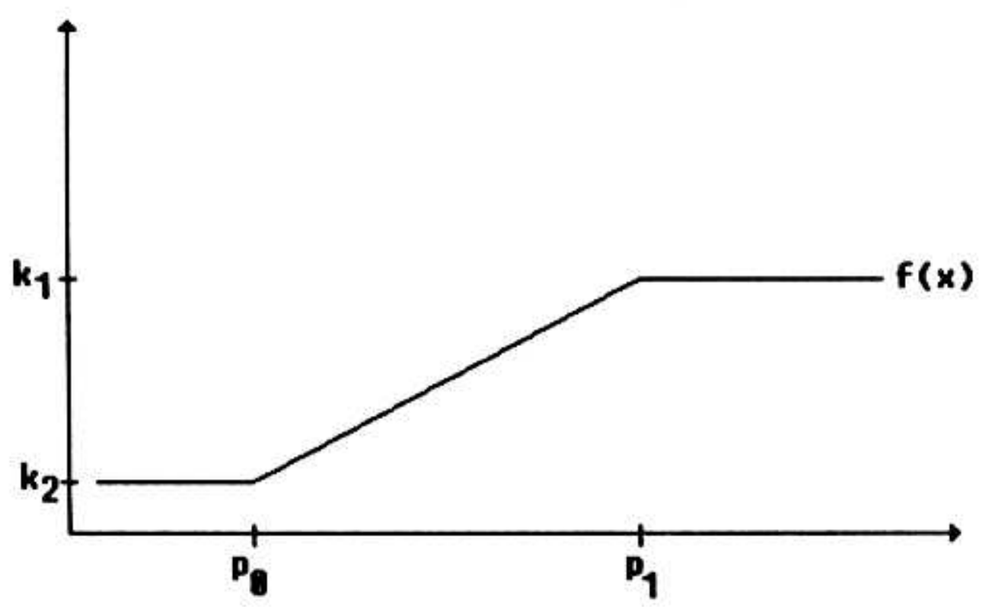
The **KERNTRACK** structure contains the data needed to generate the track-kerning algorithm. This algorithm finds the correct track-kerning values for any point size within a specified range. The algorithm follows:

$$f(x)=k_0 \qquad \text{for } x < p_0$$

$$f(x)=\frac{(k_1-k_0)}{(p_1-p_0)}x+\frac{(k_0p_1-k_1p_0)}{(p_1-p_0)} \qquad \text{for } p_0 \le x \le p_1$$

$$f(x)=k_1 \qquad \text{for } x > p_1$$

A graph of the algorithm follows. The vertical axis represents increasing track kerning. The horizontal axis represents increasing point size.



The **KERNTRACK** structure contains the following fields:

```
short degree;
short minSize;
short minAmount;
short maxSize;
short maxAmount;
```

The **KERNTRACK** fields are described below. All measurements are given in font units of the currently selected font regardless of the current mapping mode of the display context.

Field	Definition
degree	A short integer value where increasing negative values reflect increased track kerning and increasing positive values reflect decreased kerning.
minSize	A short integer value that specifies the minimum font size for which track kerning applies. (The font size is given in device units.)
minAmount	A short integer value that specifies the amount of track kerning applied to fonts whose sizes are equal to, or less than, *minSize*. (The value is given in device units.)
maxSize	A short integer value that specifies the maximum font size for which track kerning applies. (The font size is given in device units.)
maxAmount	A short integer value that specifies the amount of track kerning applied to fonts whose sizes are equal to, or greater than, *maxSize*. (The value is given in device units.)

CODE Statement
(Section 6.3, Pages 313–314)

CODE [[*segment-attributes*]]

This statement defines the attributes of the standard code segment. The standard code segment is the application segment having the name _TEXT and belonging to the class CODE. In C applications, the standard segment is created automatically if no specific segment name is given in the C-compiler command line.

Parameter	Description
segment-attributes	One or more optional keywords that specify the code-segment attributes. They can be any combination of the following:

FIXED	Segment remains at a fixed memory location.
MOVEABLE	Segment can be moved if necessary to compact memory.
DISCARDABLE	Segment can be discarded if no longer needed.
PRELOAD	Segment is loaded immediately.
LOADONCALL	Segment is loaded when called.
SHARED	Segment can be shared.
NONSHARED	Segment cannot be shared.
EXECUTEONLY	Segment can be executed only.
EXECUTEREAD	Segment can be read as data as well as executed.

Comments

If no **CODE** statement is given in the module definition file, the default attributes for the segment are **MOVEABLE**, **PRELOAD**, **NONSHARED**, and **EXECUTEREAD**.

If a **CODE** statement is given, the default attributes are **FIXED**, **LOADONCALL**, **NONSHARED**, and **EXECUTEREAD**, unless these are explicitly overridden.

If conflicting options are given in the same statement, **link4** uses the overriding option to determine the segment attributes. **MOVEABLE** overrides **FIXED**; **PRELOAD** overrides **LOADONCALL**; **SHARED** overrides **NONSHARED**; and **EXECUTEONLY** overrides **EXECUTEREAD**.

SHARED, **NONSHARED**, **EXECUTEONLY**, and **EXECUTEREAD** are used for 80286 protected-mode programs only.

PURE and **IMPURE** are alternate keywords that can be used in place of **SHARED** and **NONSHARED**, respectively.

Example

```
CODE MOVEABLE LOADONCALL
```

DATA Statement
(Section 6.3, Pages 314–315)

DATA [[*segment-attributes*]]

This statement defines the attributes of the standard data segment. The standard data segment is all application segments belonging to the group DGROUP and the class DATA. In C applications, the standard data segment is created automatically.

Parameter	Description
segment-attributes	One or more optional keywords that specify the attributes of the data segment. They can be any combination of the following:

NONE	No data segment.
SINGLE	A single segment to be shared by all instances of the module (valid only for library modules).
MULTIPLE	One segment for each instance.
FIXED	Segment remains at a fixed memory location.
MOVEABLE	Segment can be moved if necessary to compact memory.
DISCARDABLE	Segment can be discarded if no longer needed.
PRELOAD	Segment is loaded immediately.
LOADONCALL	Segment is loaded when accessed.
SHARED	Segment contains data that does not change during execution.
NONSHARED	Segment contains data that may change during execution.
READONLY	Segment contents can be read only.
READWRITE	Segment contents can be read and modified.

Comments

If no **DATA** segment is given in the module definition file, the default attributes for the segment are **MULTIPLE**, **MOVEABLE**, **PRELOAD**, **NONSHARED**, and **READWRITE**.

If a **DATA** statement is given, the default attributes are **MULTIPLE**, **FIXED**, **LOADONCALL**, **NONSHARED**, and **READWRITE**, unless these are explicitly overridden.

If conflicting options are given in the same statement, **link4** uses the overriding option to determine the segment attributes. **MULTIPLE** overrides **NONE** and **SINGLE**; **SINGLE** overrides **NONE**; **MOVEABLE** overrides **FIXED**; **PRELOAD** overrides **LOADONCALL**; **SHARED** overrides **NONSHARED**; and **READONLY** overrides **READWRITE**.

The **SINGLE** option implies **SHARED**; **MULTIPLE** implies **NONSHARED**, and vice versa. **Link4** ignores **SHARED** if it is used with **MULTIPLE** and ignores **NONSHARED** if it is used with **SINGLE**.

SHARED, **NONSHARED**, **READONLY**, and **READWRITE** are used for 80286 protected-mode programs only.

PURE and **IMPURE** are alternate keywords that can be used in place of **SHARED** and **NONSHARED**, respectively.

Example

```
DATA MOVEABLE SINGLE
```

SEGMENTS Statement
(Section 6.3, Page 315–316)

SEGMENTS *segmentname* [[**CLASS** '*class-name*']] [[*minalloc*]] [[*segment-attributes*]]

This statement defines the segment attributes of additional code and data segments. It is used for segments belonging to medium, compact, or large model memory applications.

Parameter	Description
segmentname	A character string that names the new segment. It can be any name, including the standard segment names _TEXT and _DATA that represent the standard code and data segments.
CLASS '*class-name*'	An optional keyword that specifies the class name of the given segment. If no class name is given, **link4** assumes the class name CODE by default.

minalloc — An integer number that specifies the minimum allocation size for the segment.

segment-attributes — One or more optional keywords that specify the attributes of the given segment. They can be any combination of the following:

FIXED	Segment remains at a fixed memory location.
MOVEABLE	Segment can be moved if necessary to compact memory.
DISCARDABLE	Segment can be discarded if no longer needed.
SHARED	Segment can be shared.
NONSHARED	Segment cannot be shared.
PRELOAD	Segment is loaded immediately.
LOADONCALL	Segment is loaded when accessed or called.
EXECUTEONLY	Segment can be executed only.
EXECUTEREAD	Segment can be read as data as well as executed.
READONLY	Segment contents can be read only.
READWRITE	Segment contents can be read and modified.

Comments

If no **SEGMENTS** statement is given in the module definition file, the default attributes for nonstandard segments are **MOVEABLE**, **PRELOAD**, **NONSHARED**, and **EXECUTEREAD**.

If a **SEGMENTS** statement is given but only a segment name and class are given, the default attributes are **FIXED**, **LOADONCALL**, **NONSHARED**, and **EXECUTEREAD** or **READWRITE**, unless these are explicitly overridden.

If conflicting options are given in the same statement, **link4** uses the overriding option to determine the segment attributes. **MOVEABLE** overrides **FIXED**; **PRELOAD** overrides **LOADONCALL**; **SHARED** overrides **NONSHARED**; **EXECUTEONLY** overrides **EXECUTEREAD**; and **READONLY** overrides **READWRITE**.

SHARED, **NONSHARED**, **EXECUTEONLY**, and **EXECUTEREAD** are used for 80286 protected-mode programs only.

PURE and **IMPURE** are alternate keywords that can be used in place of **SHARED** and **NONSHARED**, respectively.

Example

```
SEGMENTS
        _TEXT FIXED
        _INIT PRELOAD DISCARDABLE
        _RES  CLASS 'DATA' PRELOAD DISCARDABLE
```

STUB Statement
(Section 6.3, Page 318)

If the file named by *filename* is not in the current directory, the linker searches for the file in the directories specifies by the user's PATH environment variable.

Windows Section
(Section 6.4.1, Page 320)

The **Run** field must not contain the name of any bad applications. A bad application is any non-Windows application that requires Windows to exit memory before it can run. Placing such an application in this field causes Windows to enter an endless loop.

Ports Section
(Section 6.4.2, page 321)

The correct syntax for the section entry is the following:

portname:=*baud-rate*, *parity*, *word-length*, *stop-bits* [,**p**]

The **p** option is the retry option, which specifies that hardware handshaking is in effect.

Extensions Section
(Section 6.4, New)

The extensions section is a list of filename extensions and corresponding applications that will be invoked when the user double-clicks a file having one of the given extensions. The syntax for the extensions section follows:

[extensions]
filename-extension*=*executable-filename input-filename

The *filename-extension* is a one- to three-character filename extension; the *executable-filename* is the filename (with *.exe* extension) of the corresponding application; and the *input-filename* is the filename that will be passed to the application when invoked. The *input-filename* can be any filename or can consist of the special wildcard character "^" followed by a filename extension. The caret (^) represents the filename part of the file that the user double-clicked.

Example

```
[extensions]
doc=write.exe ^.doc
txt=notepad.exe ^.txt
asm=notepad.exe ^.txt
obj=notepad.exe readme.txt
```

Assembly-Language Macros
(Chapter 7, Pages 325–348)

ExternX Statement
(Section 7.4.1, Page 336)

The *Purpose* section for the **externX** macro should be the following:

This macro defines one or more names to be the labels of external variables or functions.

LabelX Statement
(Section 7.4.1, Pages 336–337)

The *Purpose* section for the **labelX** macro should read as follows:

This macro defines one or more names to be the labels of public (global) variables or functions.

Window Messages
(Chapter 8, Pages 349–408)

WM_SIZEWAIT Message
(Section 8.1, Internal)

Although the WM_SIZEWAIT message is defined in *windows.h*, the message is used internally by Windows and is not available to Windows applications.

WM_SYSTIMER Message
(Section 8.1, Internal)

Although the WM_SYSTIMER message is defined in *windows.h*, the message is used internally by Windows and is not available to Windows applications.

WM_CTLCOLOR Message
(Section 8.2, Page 358)

If an application processes the WM_CTLCOLOR message, it must return a handle to the brush to be used for painting the control background. Failure to return a valid brush handle will cause a serious error.

The list of values under the *Parameter* section should read as follows:

Value	Type of Input
CTLCOLOR_MSGBOX	Message box
CTLCOLOR_EDIT	Edit control
CTLCOLOR_LISTBOX	List-box control
CTLCOLOR_BTN	Button control
CTLCOLOR_DLG	Dialog box
CTLCOLOR_SCROLLBAR	Scroll-bar control
CTLCOLOR_STATIC	Static control

WM_SETREDRAW Message
(Section 8.2, Pages 359–360)

Although the WM_SETREDRAW message can be used to set the redraw flag for a list box, it does not direct the list box to update its display. Applications that need to carry out the operation given in the cited example (add several names to the list without redrawing until the last name is added) must set the redraw flag before adding the last name to the list. An alternate method is to call the **InvalidateRect** function for the list box after adding the last name and setting the redraw flag.

WM_GETDLGCODE Message
(Section 8.2, Page 360)

This message is not sent to an application's main window procedure. The message will be sent only to the input procedure associated with a control.

The list of values under the *Return Value* section should read as follows:

Value	Type of Input
DLGC_WANTARROWS	DIRECTION keys
DLGC_WANTTAB	TAB key
DLGC_WANTALLKEYS	All keyboard input
DLGC_HASSETSEL	EM_SETSEL messages

WM_CANCELMODE Message
(Section 8.2, New)

The WM_CANCELMODE message cancels any mode the system is in, such as tracking the mouse in a scroll bar or moving a window. Windows sends the message when an application displays a message box. The *wParam* and *lParam* parameters are not used.

WM_INITDIALOG Message
(Section 8.3, Pages 363–364)

The dialog function may return FALSE only if it has set the input focus to one of the controls in the dialog box.

WM_KEYDOWN Message
(Section 8.4, Pages 370–371)

The *lParam* bit fields are numbered from 1 to 32. The bit positions for the context code, previous key state, and transition-state fields are 30, 31, and 32, respectively.

WM_KEYUP Message
(Section 8.4, Pages 371–372)

The *lParam* bit fields are numbered from 1 to 32. The bit positions for the context code, previous key state, and transition-state fields are 30, 31, and 32, respectively.

WM_CHAR Message
(Section 8.4, Pages 372–373)

The *lParam* bit fields are numbered from 1 to 32. The bit positions for the context code, previous key state, and transition-state fields are 30, 31, and 32, respectively.

WM_DEADCHAR Message
(Section 8.4, Pages 373–374)

The *lParam* bit fields are numbered from 1 to 32. The bit positions for the context code, previous key state, and transition-state fields are 30, 31, and 32, respectively.

WM_COMMAND Message
(Section 8.4, Pages 374–375)

In the *lParam* description under *Parameter*, if the message is from a control, the high-order word of *lParam* contains a notification code and the low-order word contains the handle of the control that is sending the message.

If the message is an accelerator message, the high-order word is 1, and the low-order word is zero.

WM_VSCROLL Message
(Section 8.4, Pages 375–376)

If the message is sent by a scroll-bar control, the high-order word of *lParam* contains the window handle of the control. If the message is sent as a result of the user's clicking a tiled window's scroll bar, the high-order word is not used.

If *wParam* is set to EM_SCROLL and the WM_VSCROLL message is sent to a multiple-line edit control, the control returns the current thumb position of its vertical scroll bar.

WM_HSCROLL Message
(Section 8.4, Page 376)

If the *wParam* parameter is set to EM_SCROLL and the WM_HSCROLL message is sent to a multiple-line edit control, the control returns the current thumb position of its horizontal scroll bar.

If the message is sent by a scroll-bar control, the high-order word of *lParam* contains the window handle of the control. If the message is sent as a result of the user's clicking a tiled window's scroll bar, the high-order word is not used.

WM_SYSKEYDOWN Message
(Section 8.5, Pages 377–378)

The *lParam* bit fields are numbered from 1 to 32. The bit positions for the context code, previous key state, and transition-state fields are 30, 31, and 32, respectively.

WM_SYSKEYUP Message
(Section 8.5, Pages 378–379)

The *lParam* bit fields are numbered from 1 to 32. The bit positions for the context code, previous key state, and transition-state fields are 30, 31, and 32, respectively.

WM_SYSCHAR Message
(Section 8.5, Page 379)

The *lParam* bit fields are numbered from 1 to 32. The bit positions for the context code, previous key state, and transition-state fields are 30, 31, and 32, respectively.

WM_SYSCOMMAND Message
(Section 8.5, Pages 380–381)

If a System-menu command is chosen with the mouse, the *lParam* parameter contains the *x*- and *y*-coordinates of the mouse cursor. Otherwise, the parameter is not used. If a mouse coordinate is given, the low-order word is the *x*-coordinate; the high-order word is the *y*-coordinate.

EM_SETHANDLE Message
(Section 8.8.2, Pages 391–392)

If the EM_SETHANDLE message is used to change the text buffer used by an edit control, the previous text buffer is not destroyed. It is the application's responsiblity to retrieve the old buffer handle before setting the new handle and to free the old handle by using the **LocalFree** function.

An edit control automatically reallocates the given buffer whenever it needs additional space for text or removes enough text so that additional space is no longer needed.

EM_GETHANDLE Message
(Section 8.8.2, Page 392)

The EM_GETHANDLE message does return a value. It returns the data handle of the buffer used to hold the contents of the edit control.

EM_SCROLL Message
(Section 8.8.2, Page 393)

EM_SCROLL is not a message. It must not be used to direct scrolling in edit controls.

EM_LINELENGTH Message
(Section 8.8.2, Page 394)

The *wParam* parameter should be the character index of a character in the desired line, not the line number. If desired, the EM_LINEINDEX message can be used to retrieve a character index using a line number. This character index can then be used with the EM_LINELENGTH message as in the following example.

Example

```
SendMessage(hwnd,EM_LINELENGTH,
        SendMessage(hwnd,EM_LINEINDEX,iLine,0L),0L);
```

EM_GETMODIFY Message
(Section 8.8.2, New)

The EM_GETMODIFY message retrieves the current value of the modify flag for the given edit control. The control sets the flag if the user enters or modifies text in the control. The *wParam* and *lParam* parameters are not used.

EM_SETMODIFY Message
(Section 8.8.2, New)

The EM_SETMODIFY message sets the modify flag for the given edit control. The *wParam* parameter specifies the value to set the flag. The *lParam* parameter is not used.

LB_SETSEL Message
(Section 8.8.3, Page 398)

To select a string in a multiple-selection list box, the LB_SETSEL message should be used. An application that needs to remove the selection from a string in a multiple-selection list box should explicitly specify that string's index in the *lParam* parameter of the LB_SETSEL message. To remove the selection from all strings, *lParam* should be set to –1.

LB_SETCURSEL Message
(Section 8.8.3, Page 398)

The LB_SETCURSEL message cannot be used to set or remove a selection in a multiple-selection list box. Specifically, the *lParam* parameter is not used by the LB_SETCURSEL message, so setting it to –1 and attempting to remove all selections from a multiple-selection list box will fail. The message LB_SETCURSEL does remove the selection from a single-selection list box if *wParam* is –1.

LB_RESETCONTENT Message
(Section 8.8.3, New)

The LB_RESETCONTENT message removes all strings from the list box and frees any memory allocated for those strings. The *wParam* and *lParam* parameters are not used. There is no return value.

Programming Guide

Using GDI in Applications 71
Creating Windows Applications 71
Controls and Dialog Boxes 72
Using System Resources 74
The Clipboard 74

Using GDI in Applications
(Chapter 3, Pages 43–78)

Painting Shapes
(Section 3.5.1, Pages 54–55)

Delete the following statement:

After drawing, **ShapesPaint** validates the window (indicating that it has been painted) using the **ValidateRect** function.

Creating Windows Applications
(Chapter 5, Pages 101–124)

Checking the Queue
(Section 5.4.5, Page 108)

In the cited example, the call to **PeekMessage** is missing a value for the *bRemoveMsg* parameter. The call should be as follows:

```
if (PeekMessage((LPMSG)&msg, (HWND)NULL, 0, 0, FALSE)) {
```

The value FALSE ensures that no message is removed from the queue.

Removing Messages
(Section 5.4.7, Page 109)

In the cited example, the call to **PeekMessage** is missing a value for the *hWnd* parameter. The call should be:

```
while (PeekMessage((LPMSG)&msg, (HWND)NULL,
        WM_KEYFIRST, WM_KEYLAST, TRUE))
```

Notice that the last parameter, TRUE, directs the function to remove messages from the queue.

Making a Function Instance
(Section 5.6.8, Pages 121–122)

You must export any function that you will use to make a function instance. You export a function by using the **EXPORTS** statement in the module definition file.

Exported functions cannot be called directly by the application. You must make a function instance and call the function using the address returned for that instance. For example, assume that the following function is exported:

```
BOOL FAR PASCAL AbortProc( )
{
}
```

To make a function instance, you must call the **MakeProcInstance** function, as shown in the following example:

```
pAbortProc = MakeProcInstance(AbortProc, hInstance);
```

The pointer **pAbortProc** is assumed to have been declared with the **FARPROC** type. If you want to call **AbortProc**, you must not do so directly. Instead, use the following pointer:

```
(*pAbortProc)( );
```

Controls and Dialog Boxes
(Chapter 7, Pages 147–174)

Defining a Group Box
(Section 7.3.5, Page 152)

In the cited example, replace BS_GROUP with BS_GROUPBOX. The example should read as follows:

```
CONTROL "Group Box", 5, BUTTON, BS_GROUPBOX|WS_VISIBLE|WS_GROUP,
        5,5, 75,20
```

Edit-Control Styles
(Section 7.4.1, Pages 153–154)

You can make an edit control justify its text to the right by adding the ES_MULTILINE style to the ES_RIGHT style. For example, the following statement in a dialog template creates an edit control that justifies text to the right:

```
edittext IDDTEXT, 28, 31, 40, 12, ES_RIGHT |ES_MULTILINE
```

Printing Edit-Control Text
(Section 7.4.5, Page 156)

The EM_FMTLINES message, mentioned in the third paragraph, adds "soft" line breaks to the edit-control text. A soft line break consists of two carriage-return characters followed by a linefeed character (0D 0D 0A hexadecimal). The soft line breaks correspond to the word-wrapping breaks used by the control. Normally, soft line breaks are not stored in the edit-control text. They are added only after the control receives an EM_FMTLINES message.

List-Box Notification Messages
(Section 7.7.1, Page 163)

If the mouse button is pressed within a list box but released outside the box, the selection remains unchanged. If the button is released within the box but beyond the end of a string, the selection remains unchanged, but the highlight may be removed from the current selection.

List-Box Messages
(Section 7.7.2, Pages 163–166)

The LB_SETCURSEL message cannot set or remove a selection in multiple-selection list boxes. Because the *lParam* parameter is not used by the LB_SETCURSEL message, setting the parameter to –1 and attempting to remove all selections from a multiple-selection list box will fail. The LB_SETCURSEL message does remove the selection from a single-selection list box if *wParam* is –1.

Default Response in Dialog Boxes
(Section 7.10.10, Page 172)

If the user presses the ENTER key in a modal dialog box, Windows sends a WM_COMMAND message to the dialog input function with *wParam* set to 1. If you want to associate this key stroke with the dialog box's default pushbutton, you must set that pushbutton's ID to 1.

Using System Resources
(Chapter 8, Pages 175–202)

Checking for File Existence
(Section 8.8.9, Page 193)

In the cited example, the function calls **_lopen** and **_lclose** are sample calls to be supplied by the user.

The Clipboard
(Chapter 9, Pages 203–218)

Clipboard Viewer Chain
(Section 9.9, Pages 211–212)

An application that removes a window from the clipboard chain must call the **ChangeClipboardViewer** function to inform all windows in the chain that a change is taking place. If the window being removed is not the first in the chain, the **ChangeClipboardChain** sends the message WM_CHANGECBCHAIN to the windows in the chain. To preserve the clipboard viewer chain, a window that receives WM_CHANGECBCHAIN as a message should either process it or pass it on to the next window in the chain. The following example shows how to process the message:

```
case WM_CHANGECBCHAIN:
    if (hwndNext == (HWND)wParam) {
        hwndNext = (HWND)LOWORD(lParam);
    } else {
        SendMessage(hwndNext,WM_CHANGECBCHAIN,wParam,lParam);
    }
    break;
```

Each window in the chain must keep the handle of the next window in the chain. In the example above, the window stores this handle in the variable *hwndNext*. A window changes this value only if the message WM_CHANGECBCHAIN identifies that window as the one being removed from the chain.